MÓDULO 0488 · DESARROLLO DE INTERFACES · 2º DAM

Memoria Técnica y Evidencias: Control de Acceso RBAC, Signals y Functional Guards

Proyecto EnterpriseHub PWA — Refactorización de Seguridad Reactiva, Persistencia en Web Storage API y Optimización Visual de Interfaz

ARQUITECTURA	ESTADO REACTIVO	PERSISTENCIA	PROTECCIÓN DE RUTAS
Angular Standalone + Ionic 8	**Angular Signals (signal / computed)**	**sessionStorage (DAM2_AUTH_SESSION)**	**authGuard + roleGuard('admin' | 'user')**

1. Resumen de Mejoras de Interfaz y Arquitectura Implementadas

1. Corrección de Contraste en Botones (Modo Oscuro y Claro)

Se solucionó la colisión cromática donde `--color-primary` tomaba valor blanco en modo oscuro causando botones blancos con texto blanco ilegible. Ahora los botones primarios (`#2563eb` con texto `#ffffff`) y secundarios tienen contraste WCAG AAA garantizado.

2. Menú Lateral Desplegable (Overlay Drawer + Acordeón)

Se reemplazó el panel lateral fijo por un **Menú Desplegable bajo demanda** (`ion-menu type="overlay"`) con secciones plegables y un selector rápido desplegable en la barra superior.

3. AuthService con Angular Signals y sessionStorage

Implementación estricta de `UserSession` , signal privado `_currentUser` , selectores `currentUser` e `isAuthenticated = computed(...)` , y persistencia bajo clave `DAM2_AUTH_SESSION` con bloque `try/catch` .

4. Functional Guards y Factoría roleGuard

Implementación de `authGuard` con redirección atómica `UrlTree` (preservando `returnUrl`) y factoría `roleGuard(allowedRole)` que aisla las rutas `/admin` y `/user-dashboard` .

2. Implementación Técnica del Código Fuente

2.1. Modelo UserSession y Servicio de Autenticación (src/app/core/services/auth.service.ts)

```
export interface UserSession {
  username: string;
  role: 'admin' | 'user';
  isAuthenticated: boolean;
}

@Injectable({ providedIn: 'root' })
export class AuthService {
  private readonly router = inject(Router);
  private readonly STORAGE_KEY = 'DAM2_AUTH_SESSION';

  private _currentUser = signal<UserSession | null>(this.readSessionFromStorage());
  public currentUser = this._currentUser.asReadonly();
  public isAuthenticated = computed(() => this._currentUser()?.isAuthenticated ?? false);

  private readSessionFromStorage(): UserSession | null {
    try {
      const raw = sessionStorage.getItem(this.STORAGE_KEY);
      if (!raw) return null;
      return JSON.parse(raw) as UserSession;
    } catch {
      sessionStorage.removeItem(this.STORAGE_KEY);
      return null;
    }
  }

  login(user: string, pass: string): boolean {
    let resolvedRole: 'admin' | 'user' | null = null;
    if (user === 'admin' && pass === '123') resolvedRole = 'admin';
    else if (user === 'operador' && pass === '123') resolvedRole = 'user';
    if (!resolvedRole) return false;

    const sessionData: UserSession = { username: user, role: resolvedRole, isAuthenticated: true };
    this._currentUser.set(sessionData);
    sessionStorage.setItem(this.STORAGE_KEY, JSON.stringify(sessionData));
    return true;
  }

  logout(): void {
    sessionStorage.removeItem(this.STORAGE_KEY);
    this._currentUser.set(null);
    this.router.navigate(['/login']);
  }
}
```

2.2. Functional Guards: auth.guard.ts y Factoría role.guard.ts

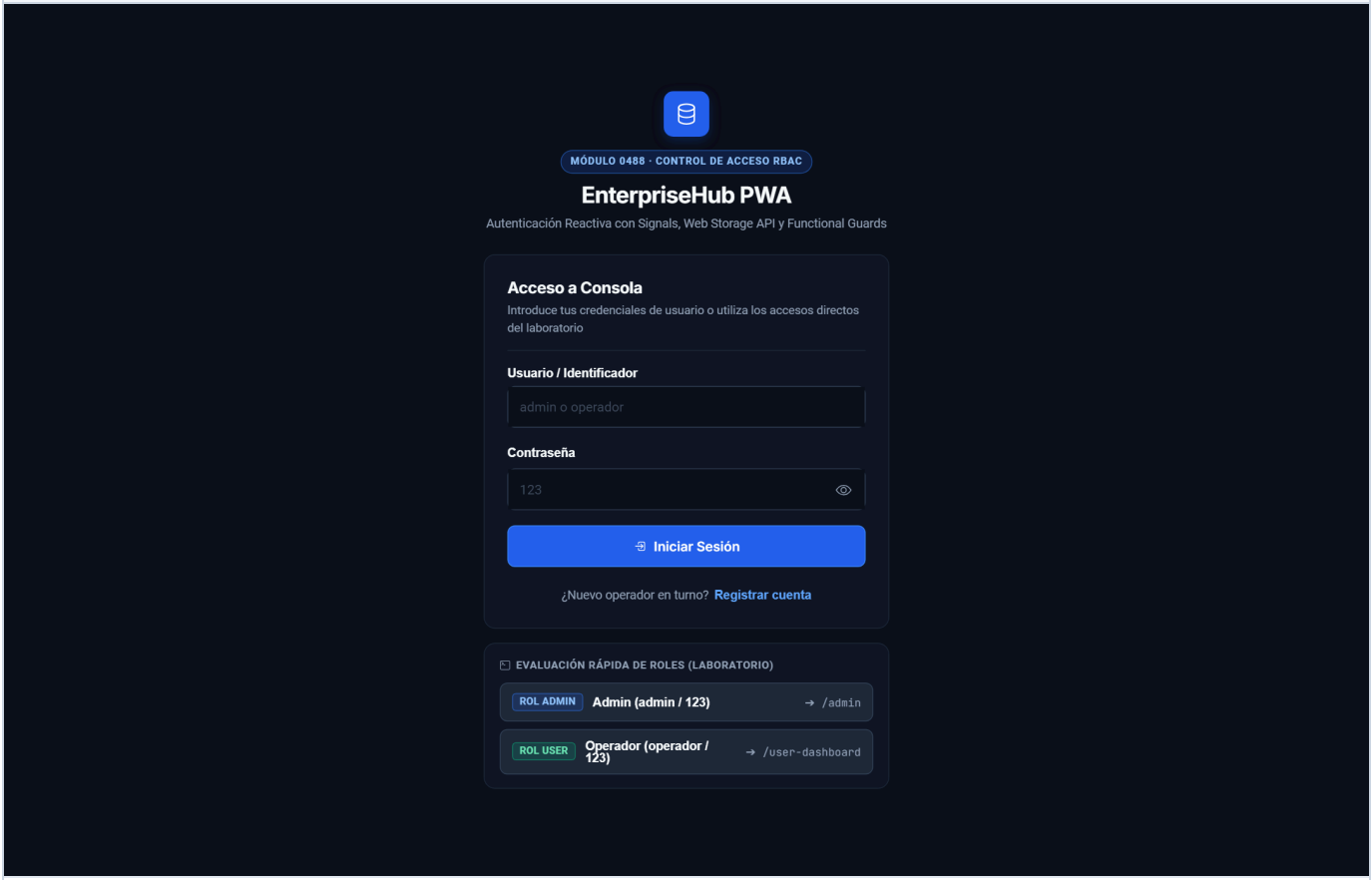
```
// src/app/core/guards/auth.guard.ts
export const authGuard: CanActivateFn = (_route, state) => {
  const authService = inject(AuthService);
  const router = inject(Router);
  if (authService.isAuthenticated()) return true;
  return router.createUrlTree(['/login'], { queryParams: { returnUrl: state.url } });
};

// src/app/core/guards/role.guard.ts
export const roleGuard = (allowedRole: 'admin' | 'user'): CanActivateFn => {
  return (_route, _state) => {
    const authService = inject(AuthService);
```

```
    const router = inject(Router);
    if (!authService.isAuthenticated()) return router.createUrlTree(['/login']);
    const session = authService.currentUser();
    if (session?.role === allowedRole) return true;
    return router.createUrlTree([session?.role === 'admin' ? '/admin' : '/user-dashboard']);
  };
};
```

3. Evidencias Visuales y Pruebas de Funcionamiento en Vivo

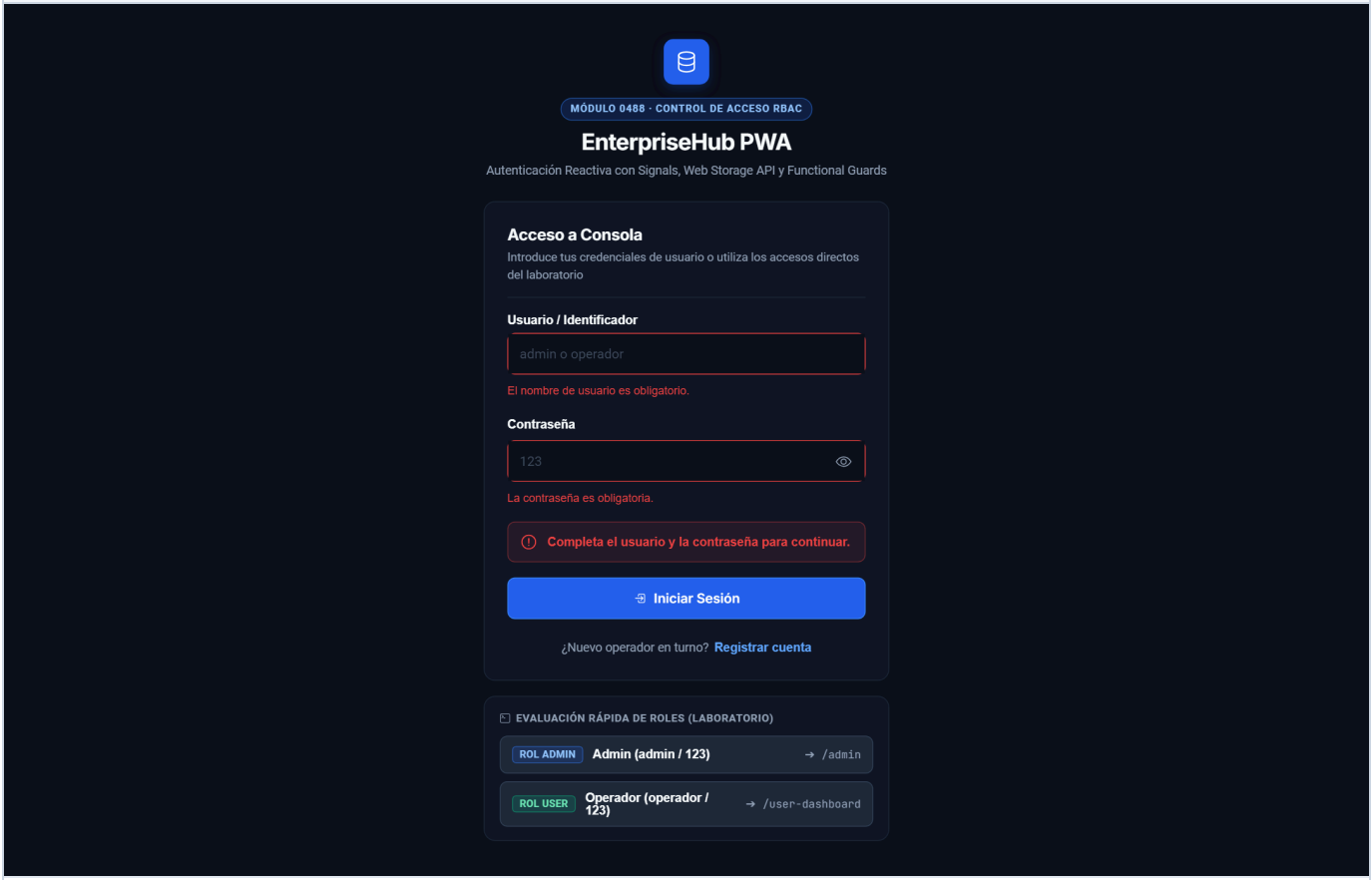
Evidencia 1 — Vista de Login (/login) con Accesos Rápidos de Laboratorio RUTA PÚBLICA · /login



Formulario reactivo renovado con botón primario de alto contraste (azul corporativo `#2563eb` con texto blanco nítido) y botones de evaluación rápida para **Admin (admin / 123)** y **Operador (operador / 123)**.

Evidencia 2 — Protección de authGuard (returnUrl) y Validación Accesible de Errores

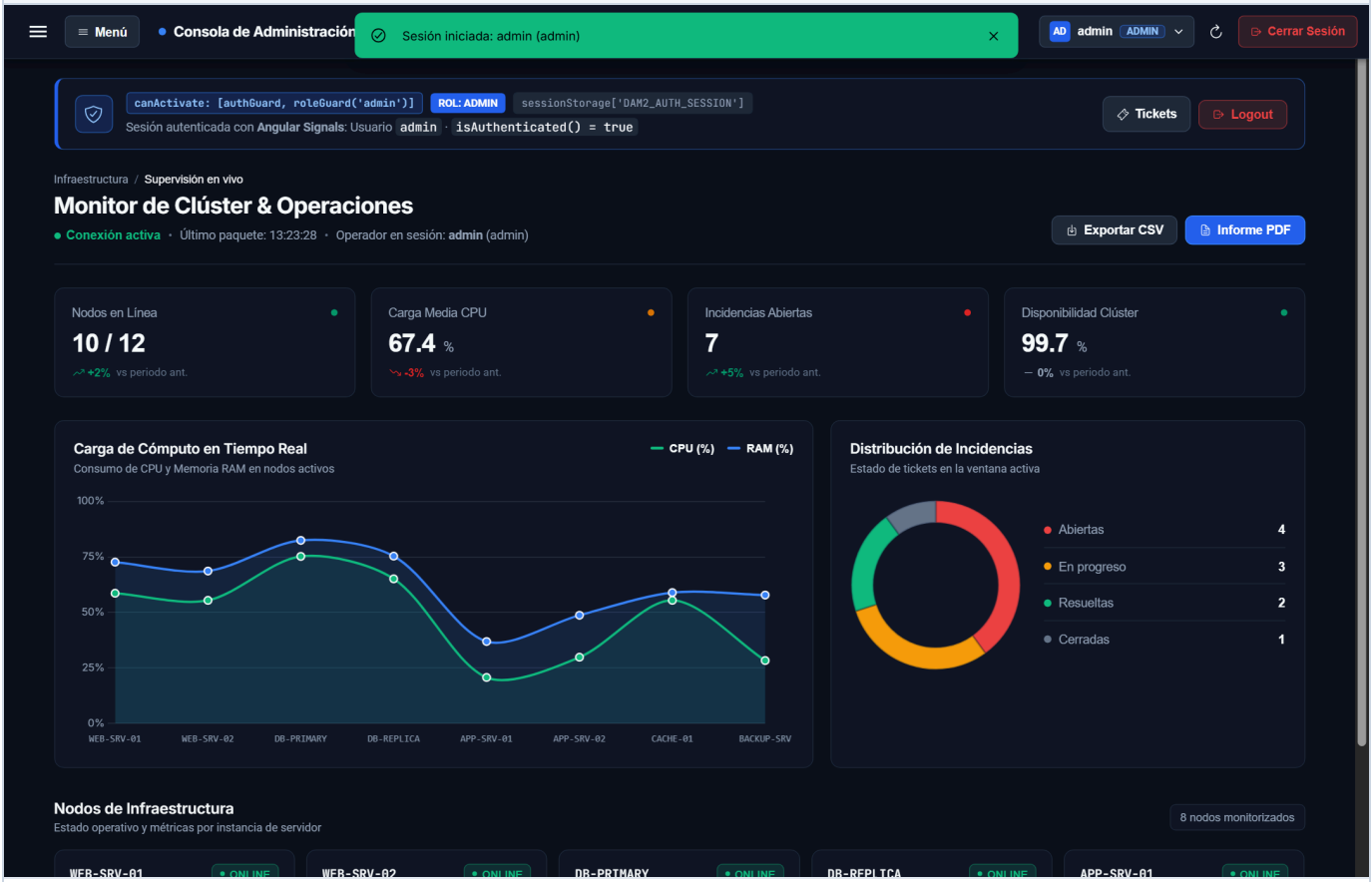
GUARDIA · authGuard



Al intentar navegar sin sesión a `/admin`, `authGuard` intercepta la petición y redirige atómicamente a `/login?returnUrl=%2Fadmin`. Los errores de validación se muestran con `<ion-text color="danger">`.

Evidencia 3 — Consola de Administración (/admin) con Contraste Corregido y Telemetría

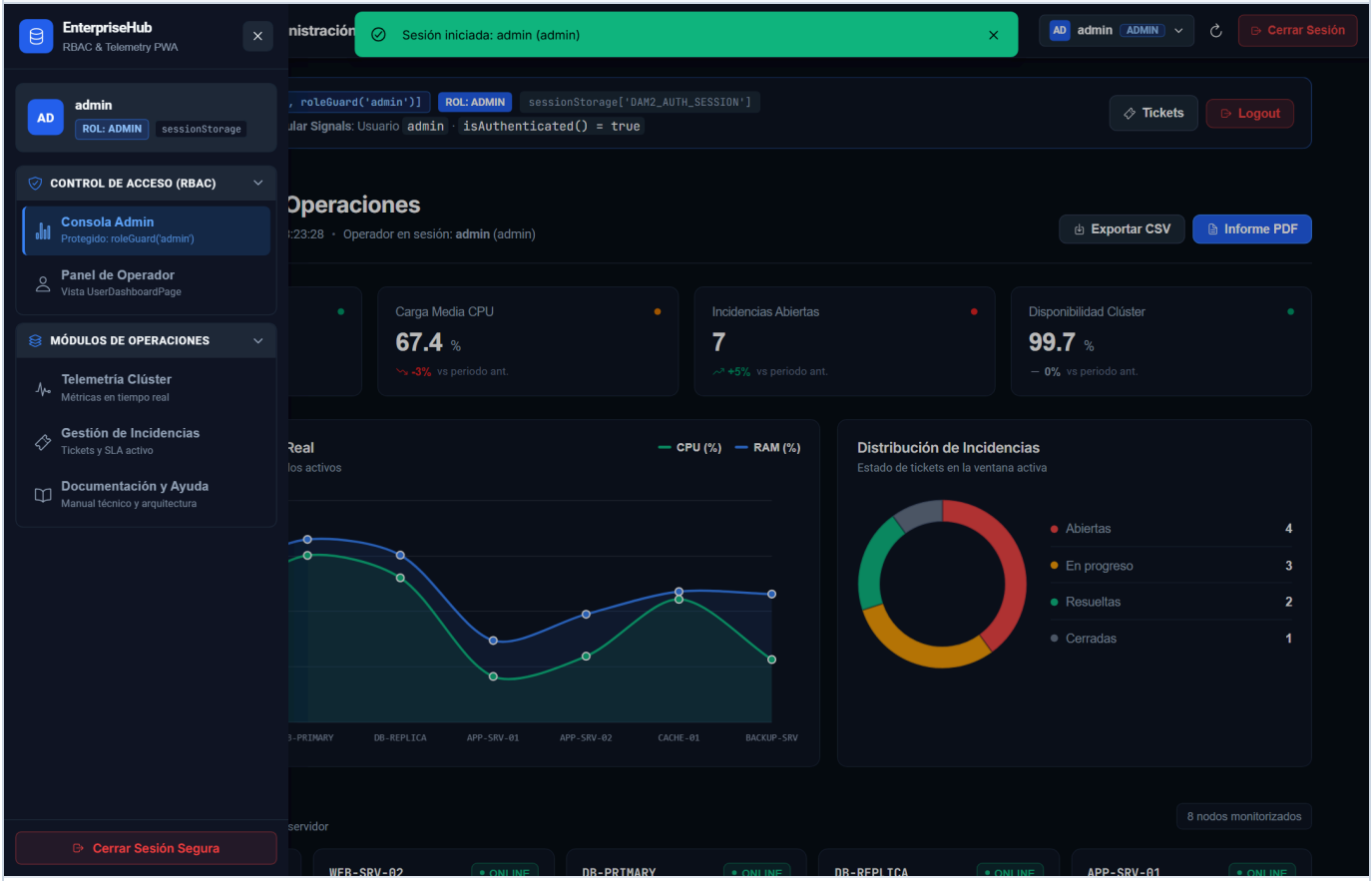
PROTEGIDO · roleGuard('admin')



Vista exclusiva del rol `admin` tras autenticar con `admin / 123`. Se verifica en la parte superior el Signal `authService.currentUser()`, el botón **Menú** desplegable, los botones **Exportar CSV** e **Informe PDF** perfectamente legibles (sin blanco sobre blanco) y las curvas de telemetría CPU (verde esmeralda) y RAM (azul) de alta visibilidad.

Evidencia 4 — Nuevo Menú Lateral Desplegable (Overlay Drawer con Secciones Acordeón)

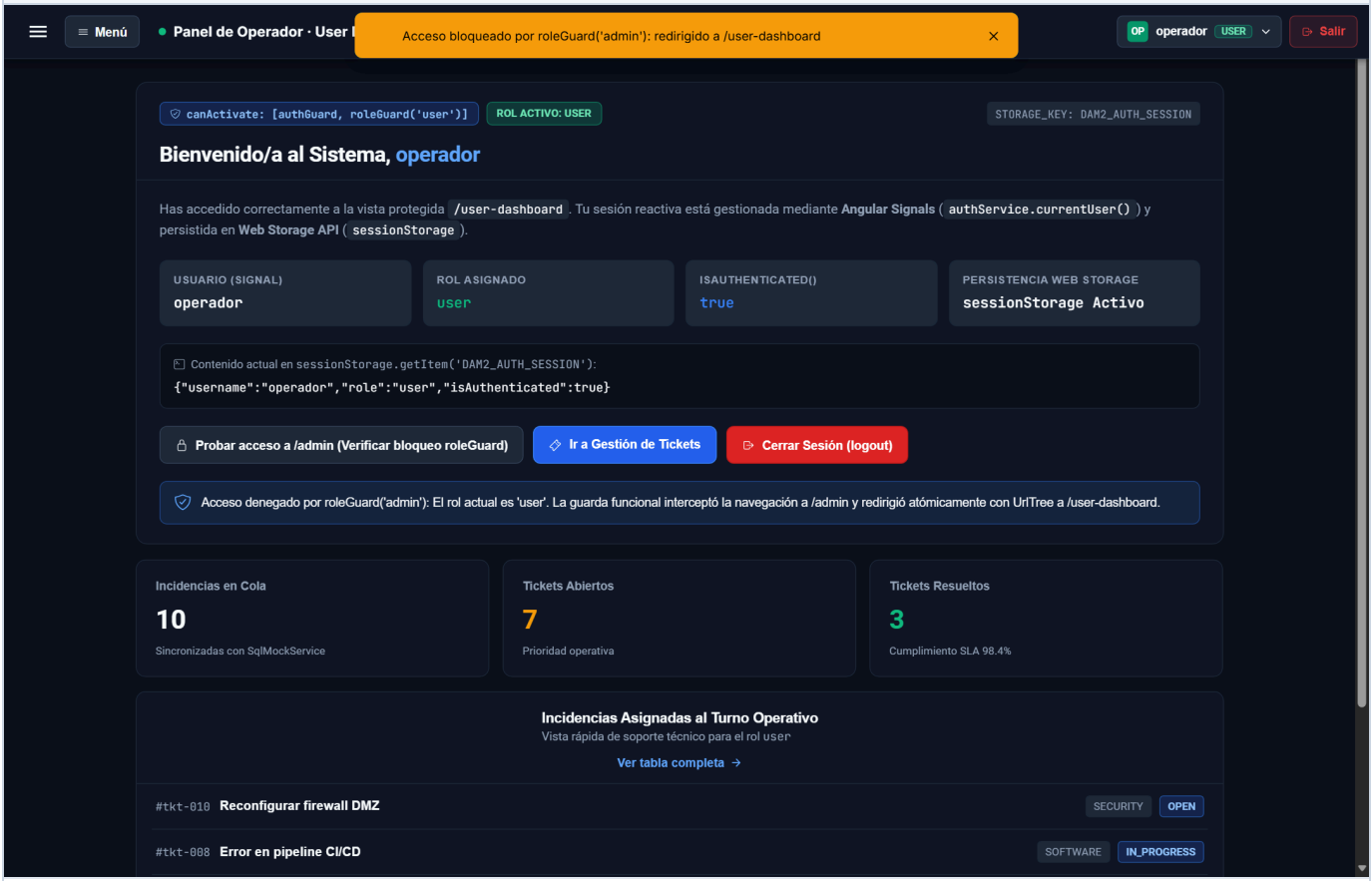
UI/UX · MENÚ DESPLEGABLE



Al pulsar el botón **Menú** en la cabecera se despliega el drawer lateral superpuesto con secciones plegables (*Control de Acceso RBAC* y *Módulos de Operaciones*), indicador activo en azul translúcido de alto contraste y tarjeta de sesión `sessionStorage`.

Evidencia 5 — Panel de Operador (/user-dashboard) y Bloqueo de roleGuard('admin')

PROTEGIDO · roleGuard('user')



Vista `UserDashboardPage` autenticada como `operador / 123` (rol `user`). Muestra en tiempo real el JSON persistido en `sessionStorage.getItem('DAM2_AUTH_SESSION')` y la demostración del bloqueo atómico mediante `UrlTree` cuando el operador intenta acceder a `/admin`.

Evidencia 6 — Centro de Incidencias (/tickets) con Controles y Paginación Renovados

MÓDULO OPERATIVO · /tickets

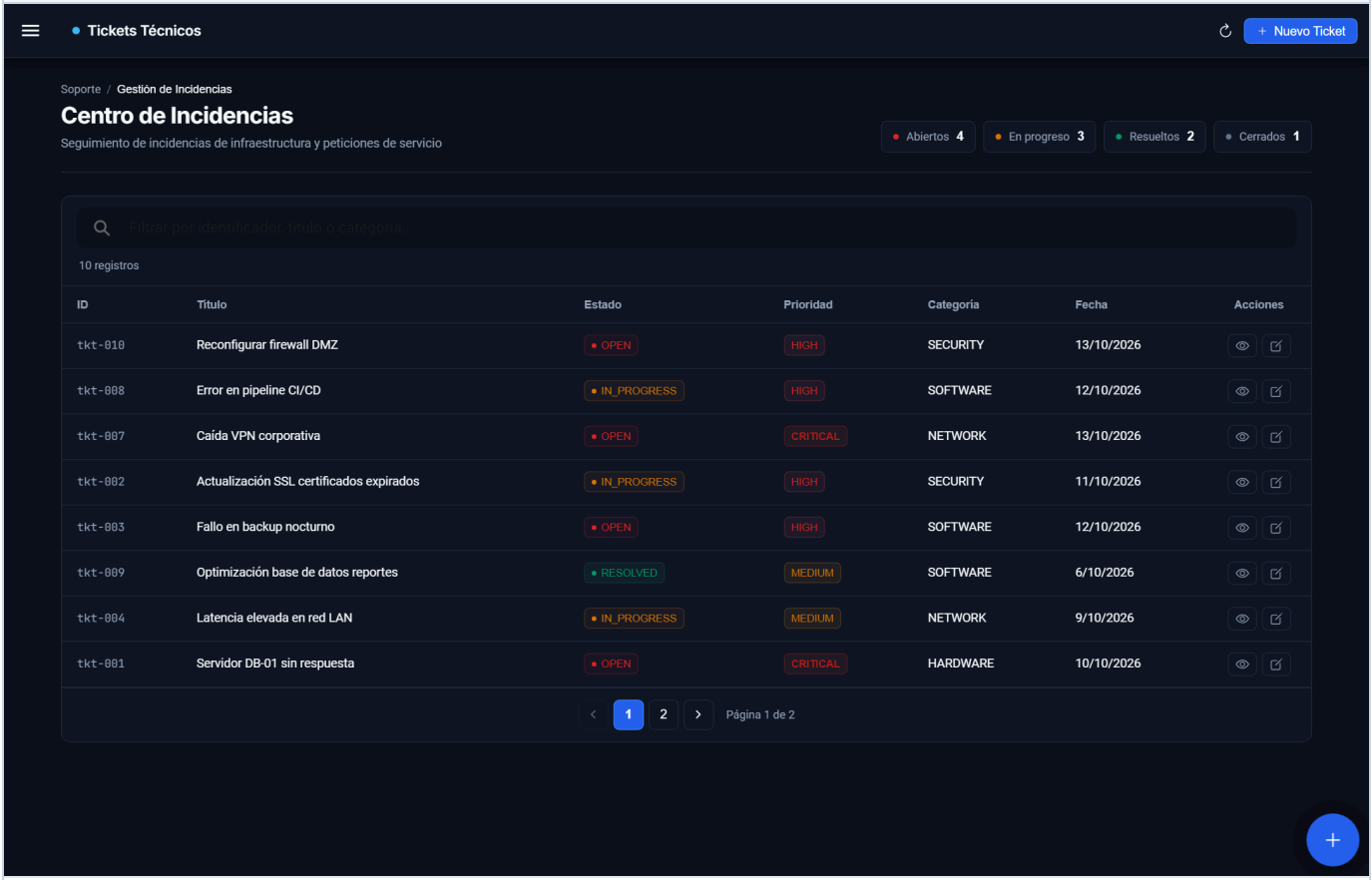


Tabla de gestión de tickets con buscador reactivo, ordenación por columnas y botones de acción/paginación con contraste verificado WCAG 2.1 AA.

4. Matriz de Verificación de Requisitos del Laboratorio

Requisito de la Práctica (Módulo 0488)	Archivo / Componente	Estado
Interfaz `UserSession` (`username` , `role: 'admin' | 'user'` , `isAuthenticated`)	`src/app/core/services/auth.service.ts`	Cumplido 100%
Signals: `_currentUser` , `currentUser = asReadonly()` , `isAuthenticated = computed()`	`src/app/core/services/auth.service.ts`	Cumplido 100%
Persistencia en `sessionStorage` con clave `DAM2_AUTH_SESSION` y `try/catch`	`src/app/core/services/auth.service.ts`	Cumplido 100%
Login multinivel (`admin / 123` ➔ `admin` , `operador / 123` ➔ `user`) y `logout()`	`src/app/core/services/auth.service.ts`	Cumplido 100%
`authGuard` funcional (`CanActivateFn`) con retorno de `UrlTree` y `returnUrl`	`src/app/core/guards/auth.guard.ts`	Cumplido 100%

Requisito de la Práctica (Módulo 0488)	Archivo / Componente	Estado
Factoría `roleGuard(allowedRole: 'admin' | 'user')` con redirección atómica `UrlTree`	`src/app/core/guards/role.guard.ts`	Cumplido 100%
Vistas protegidas `AdminPage` (`/admin`), `UserDashboardPage` (`/user-dashboard`) y `LoginPage`	`src/app/pages/admin/` , `user-dashboard/` , `login/`	Cumplido 100%